АЛГОРИТМЫ ЭЛЕКТРОННОЙ ЦИФРОВОЙ ПОДПИСИ (ЭЦП)

СВОЙСТВА ЭЦП И ОБЫЧНОЙ ПОДПИСИ

1. Подписать документ может только «законный» владелец подписи (и, следовательно, никто не может подделать подпись).

2. Автор подписи не может от нее отказаться.

3. В случае возникновения спора возможно участие третьих лиц (например, суда) для установления подлинности подписи.


Закрытый (приватный)
ключ пользователя
Открытый (публичный)
ключ пользователя
Исходный
документ
Подписание
Подписанный
документ
Сверка
Сверенный
документ

СЕМЕЙСТВА (ПОКОЛЕНИЯ) АЛГОРИТМОВ ЭЛЕКТРОННОЙ ПОДПИСИ:

- RSA
- ElGamal
- ГОСТ (государственный стандарт РФ)
- DSS (Digital Signature Standart – USA)

ЭЛЕКТРОННАЯ ЦИФРОВАЯ ПОДПИСЬ RSA

- **Цифровая подпись RSA** — это криптографический механизм, позволяющий подтвердить подлинность документа или сообщения и убедиться, что они не были изменены после подписания.



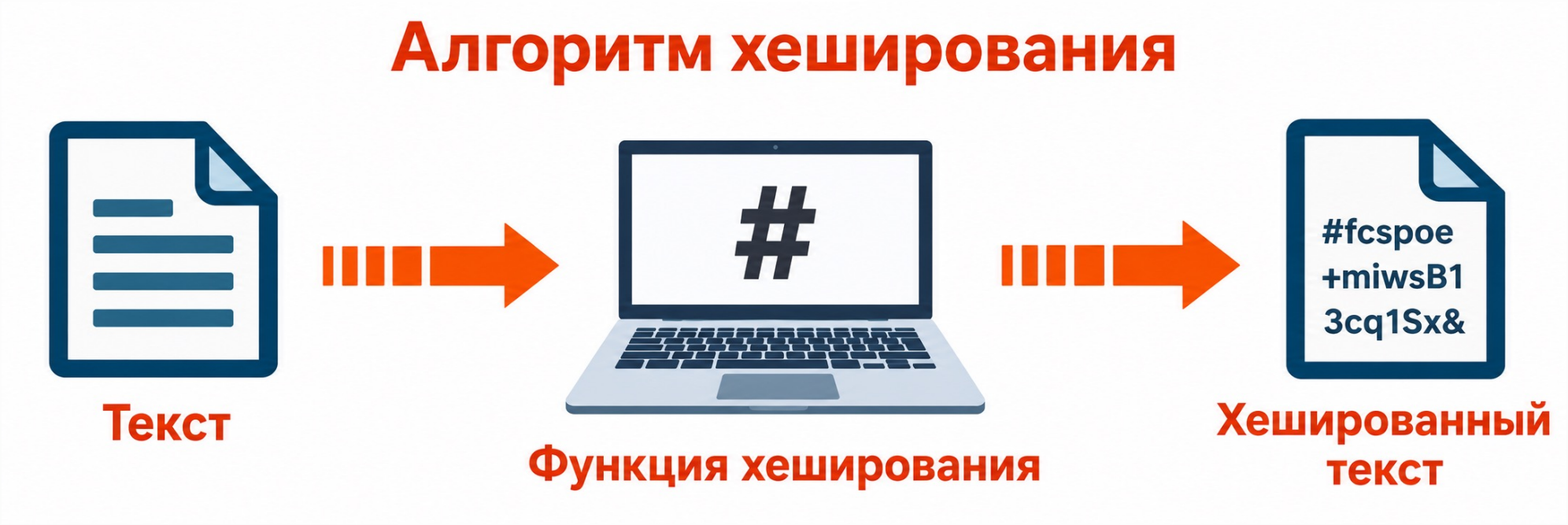
1. **Хеширование:** Сначала исходное сообщение пропускается через криптографическую хеш-функцию, чтобы получить хеш-значение фиксированной длины.
2. **Создание подписи (Signing):** Отправитель (A) подписывает полученный хеш с помощью своего **секретного (закрытого) ключа** C_A по формуле.
3. **Проверка подписи (Verification):** Получатель (B) берет **открытые ключи** отправителя (D_A , N_A) и расшифровывает подпись по формуле.

Затем получатель (B) сам вычисляет хеш от полученного сообщения и сравнивает его с расшифрованным хешем.

Если они совпадают, подпись считается верной.

КРИПТОГРАФИЧЕСКОЕ ХЕШИРОВАНИЕ

- **Хеширование** – это преобразование массива входных данных произвольной длины в битовую строку фиксированной длины с помощью специальной функции.



КРИПТОГРАФИЧЕСКИЕ ХЕШ-ФУНКЦИИ

На такие функции накладываются особые требования надёжности. К ним относятся функции семейств:

MD (MD2, MD4, MD5, MD6),

SHA (SHA-1, SHA-2, SHA-3),

ГОСТ (ГОСТ Р 34.11-94, ГОСТ Р 34.11-2012)

и другие.

ОСОБЕННОСТИ КРИПТОГРАФИЧЕСКИХ ХЕШ-ФУНКЦИЙ:

1. **Необратимость**:

для известного m вычислительно неосуществимо найти блок данных **X**, для которого **H(X) = m**. Если хеш-функция обладает данным свойством, ее также называют односторонней.

2. Стойкость к коллизиям 1 рода. При таком условии очень трудоемко (практически неосуществимо) для известного сообщения **M** подобрать другое сообщение **N**, для которого **H(M) = H(N)**.

3. Стойкость к коллизиям 2 рода. Она предполагает, что почти невозможно (очень сложно вычислительно) подобрать пару сообщений **M**, **M'**, имеющих совпадающие хеш-значения.

4. Лавинный эффект.

Он заключается в том, что при малейшем изменении аргумента значение функции сильно меняется.

5. Скорость вычисления.

Для любого заданного **X** вычисление **H(X)** должно выполняться относительно быстро.

Коллизия – ситуация, когда двум различным сообщениям соответствуют одинаковые значения хеш-функции.

Хеш-функции являются сжимающими (отображают множество в менее мощное множество), поэтому коллизии неизбежны, и становится важна вычислительная сложность их нахождения.

Коллизия – ситуация, когда двум различным сообщениям соответствуют одинаковые значения хеш-функции.

Хеш-функции являются сжимающими (отображают множество в менее мощное множество), поэтому коллизии неизбежны, и становится важна вычислительная сложность их нахождения.

Отличие коллизий **первого** и **второго рода**:

в первом случае дан конкретный исходный блок, цель - подобрать такой другой исходный блок с таким же хеш-значением (чтобы произошла коллизия);

во втором случае конкретного исходного сообщения нет, и нужно просто найти любую пару с одинаковыми хеш-значениями.

- Устойчивость к коллизиям второго рода более сильная, чем к коллизиям первого рода (если функция устойчива к коллизиям второго рода, она автоматически устойчива и к коллизиям первого рода).
- В **схеме электронной подписи** могут использоваться криптографические хеш-функции, устойчивые к коллизиям **первого рода**.

- Устойчивость к коллизиям второго рода более сильная, чем к коллизиям первого рода (если функция устойчива к коллизиям второго рода, она автоматически устойчива и к коллизиям первого рода).
- В **схеме электронной подписи** могут использоваться криптографические хеш-функции, устойчивые к коллизиям **первого рода**.
- Процесс хеширования является необратимым, что позволяет обезопасить данные от изменения злоумышленниками.
- Если даже оригинальный документ или транзакция будут в дальнейшем изменены, то в результате они получат другую цифровую подпись, что сигнализирует о несоответствии в системе.

Конструкция Меркля-Дамгарда (для создания хешей, удовлетворяющих вышеописанным свойствам) легла в основу таких хеш-функций, как:

- **MD 2, MD4, MD 5, MD 6**,
- линейка **RIPEMD**,
- **ГОСТ Р 34.11-94**,
- **Стрибог (ГОСТ Р 34.11-2012)**,
- а также поколений функций **SHA-0, SHA-1, SHA-2**.

Однако линейка функций **MD** была дискредитирована.

ПОЧЕМУ ЛИНЕЙКА MD ДИСКРЕДИТИРОВАНА?

- **Уязвимость к коллизиям:** в 2004 году китайские криптографы научились эффективно создавать два разных файла с одинаковым хешем. Появились инструменты для генерации таких пар за несколько секунд на обычном домашнем компьютере.

ПОЧЕМУ ЛИНЕЙКА MD ДИСКРЕДИТИРОВАНА?

- **Уязвимость к коллизиям:** в 2004 году китайские криптографы научились эффективно создавать два разных файла с одинаковым хешем. Появились инструменты для генерации таких пар за несколько секунд на обычном домашнем компьютере.

- **Малая длина хеша:** выходной размер MD5 составляет всего 128 бит (16 байт). Из-за этого математически ограничено число возможных вариантов, что делает алгоритм уязвимым для атак грубой силы (брутфорса) и радужных таблиц (RainbowCrack) при проверке паролей.

ПОЧЕМУ ЛИНЕЙКА MD ДИСКРЕДИТИРОВАНА?

- **Уязвимость к коллизиям:** в 2004 году китайские криптографы научились эффективно создавать два разных файла с одинаковым хешем. Появились инструменты для генерации таких пар за несколько секунд на обычном домашнем компьютере.

- **Малая длина хеша:** выходной размер MD5 составляет всего 128 бит (16 байт). Из-за этого математически ограничено число возможных вариантов, что делает алгоритм уязвимым для атак грубой силы (брутфорса) и радужных таблиц (RainbowCrack) при проверке паролей.

- **Высокая скорость работы:** быстрое вычисление хеша обернулось минусом. Современные процессоры могут перебирать миллиарды значений MD5 в секунду, позволяя легко подбирать слабые пароли.

ЧЕМ ЭТО ОПАСНО НА ПРАКТИКЕ

- **Подделка цифровых подписей и сертификатов:** злоумышленник может создать вредоносный файл или фальшивый цифровой сертификат, имеющий точно такую же хеш-сумму MD5, как у легитимного документа.

ЧЕМ ЭТО ОПАСНО НА ПРАКТИКЕ

- **Подделка цифровых подписей и сертификатов:** злоумышленник может создать вредоносный файл или фальшивый цифровой сертификат, имеющий точно такую же хеш-сумму MD5, как у легитимного документа.

- **Небезопасное хранение паролей:** использование MD5 для хеширования пользовательских паролей позволяет моментально взламывать их с помощью заранее подготовленных баз (радужных таблиц) или быстрых переборов.

РАДУЖНЫЕ ТАБЛИЦЫ (RAINBOW TABLES)

Радужные таблицы (Rainbow Tables) – это огромные заранее вычисленные базы данных, которые используются для быстрого взлома паролей, сохраненных в виде криптографических хешей.

Этот метод заменяет долгий перебор всех возможных вариантов (брутфорс) простым и быстрым поиском по готовой таблице.

РАДУЖНЫЕ ТАБЛИЦЫ (RAINBOW TABLES)

Радужные таблицы (Rainbow Tables) – это огромные заранее вычисленные базы данных, которые используются для быстрого взлома паролей, сохраненных в виде криптографических хешей.

Этот метод заменяет долгий перебор всех возможных вариантов (брутфорс) простым и быстрым поиском по готовой таблице.

Как это работает: проблема памяти и времени

- Обычный перебор (брутфорс) требует много времени, а хранение абсолютно всех цепочек «пароль → хеш» (словарей) требует терабайты или петабайты памяти.
- Радужные таблицы — это **разумный компромисс** между временем работы процессора и объемом жесткого диска.

Вместо того чтобы сохранять каждый пароль и его хеш, алгоритм генерирует длинные цепочки с помощью двух функций:

1. **Хеш-функция:** Превращает пароль в хеш

   (например, 12345 → 827ccb...).

2. **Функция редукции:** Превращает полученный хеш обратно в новый текстовый пароль (не обязательно исходный, просто случайный набор символов для продолжения цепочки).

В саму радужную таблицу записываются **только первое (исходный пароль) и последнее (финальный хеш) звенья** цепочки из тысяч шагов.

ПРОЦЕСС ВЗЛОМА

Когда злоумышленник хочет взломать украденный хеш:

- Программа берет этот хеш, применяет к нему **функцию редукции**, затем снова хеширует и проверяет, совпало ли значение с концом какой-либо цепочки в таблице.
- Если совпадение найдено, цепочка восстанавливается с самого начала (с первого звена), пока алгоритм не дойдет до искомого пароля.

КАК ЭТО УСТРОЕНО МАТЕМАТИЧЕСКИ

Представим, что мы ищем пароли, состоящие только из 4 цифр (от 0000 до 9999). Всего есть 10 000 вариантов.

Хеш MD5 — это длинная 32-значная строка в шестнадцатеричной системе (например, 827ccb0eea8a706c4c34a16891f84e7b).

Задача функции редукции R — превратить этот длинный хеш обратно в 4-значное число.

Самый простой способ сделать это — взять остаток от деления (операция mod):

Новый пароль = Числовое значение хеша mod 10000

ПОШАГОВЫЙ ПРИМЕР РАБОТЫ В ЦЕПОЧКЕ

Как функция редукции работает при создании цепочки в таблице:

1. **Старт:** Берем начальный пароль: **1234**.
2. **Хеширование (H):** Находим его хеш. Допустим, он равен **a1b2c3**....
3. **Редукция (R1):** Функция редукции берет хеш a1b2c3... и по формуле превращает его в новый текст: **5567**.
4. **Хеширование (H):** Хешируем 5567. Получаем хеш **f8e9d0**....
5. **Редукция (R2):** Вторая функция редукции превращает хеш f8e9d0... в текст: **9012**.

В радужную таблицу запишется только **начало (1234)** и **конец (9012 или его хеш)**.

[1234] -> H() -> (a1b2...) -> R1() -> [5567] -> H() -> (f8e9...) -> R2() -> [9012]

КАК ЭТО ИСПОЛЬЗУЕТСЯ ПРИ ВЗЛОМЕ

Допустим, злоумышленник украл базу данных и хочет узнать пароль для хеша **f8e9d0**....

1. Он берет этот хеш и применяет к нему функцию редукции **R1**.
2. Получает текст **9012**.
3. Проверяет радужную таблицу: «Есть ли у меня цепочка, заканчивающаяся на 9012?»
4. Таблица отвечает: **«Да, есть. Она начинается с 1234»**.
5. Программа взлома берет стартовый пароль **1234** и начинает заново строить цепочку [**хеш** → **редукция** → **хеш** ...].
6. На одном из шагов программа получает хеш **f8e9d0**.... Она смотрит, какой пароль стоял прямо перед ним. Это пароль **5567**.
7. **Пароль взломан!**

ПОЧЕМУ ТАБЛИЦА НАЗЫВАЕТСЯ «РАДУЖНОЙ»

Если на каждом шаге использовать одну и ту же функцию редукции **R**, то при совпадении текстов в разных цепочках они начнут полностью дублировать друг друга (**произойдет коллизия цепочек**).

Это сделает таблицу неэффективной!

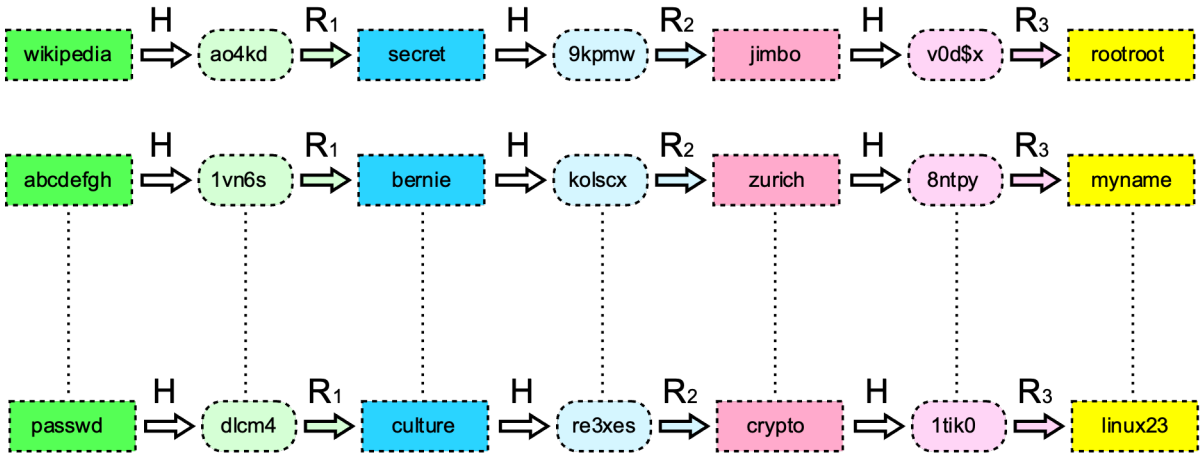
ПОЧЕМУ ТАБЛИЦА НАЗЫВАЕТСЯ «РАДУЖНОЙ»

Если на каждом шаге использовать одну и ту же функцию редукции **R**, то при совпадении текстов в разных цепочках они начнут полностью дублировать друг друга (**произойдет коллизия цепочек**).

Это сделает таблицу неэффективной!

Чтобы этого не происходило, для каждого шага в цепочке используют **свою уникальную функцию редукции** (**R1**, **R2**, **R3** …).

Каждому шагу соответствует свой «цвет» функции – поэтому алгоритм Филиппа Оэслана и получил название **«радужные таблицы»**.



ПОЧЕМУ ОНИ ЭФФЕКТИВНЫ ПРОТИВ ХЕШ-ФУНКЦИЙ

- MD5 и другие хеш-функции вычисляются очень быстро и не имеют встроенной защиты от предварительных вычислений.
- Для них созданы гигантские радужные таблицы, которые позволяют восстановить простой или средней сложности пароль по его хешу всего за несколько секунд.

КАК ЗАЩИТИТЬСЯ ОТ РАДУЖНЫХ ТАБЛИЦ?

- Использовать «Соль» (Salt)

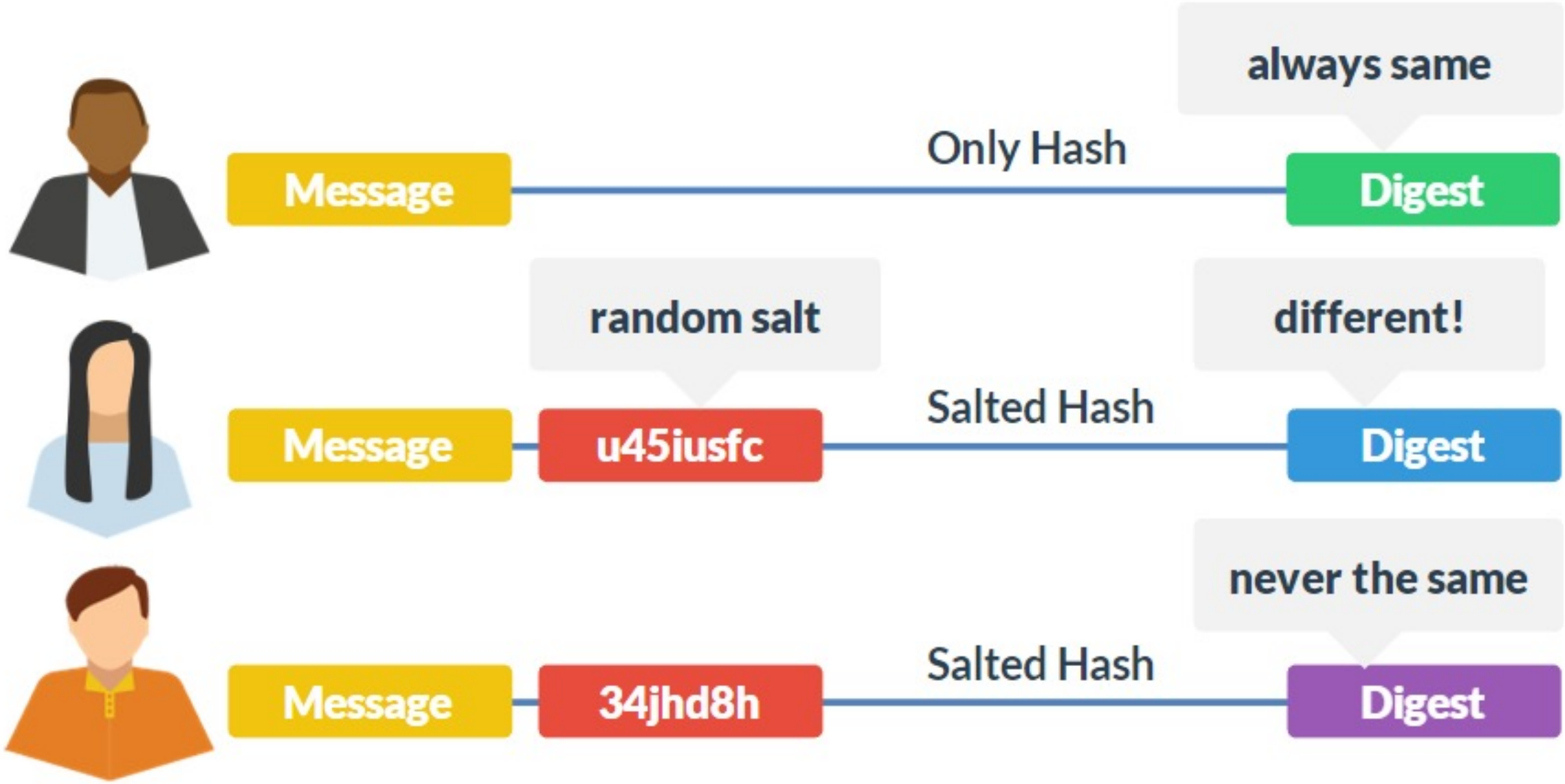
КАК ЗАЩИТИТЬСЯ ОТ РАДУЖНЫХ ТАБЛИЦ?

- Использовать **«Соль»** (Salt)

Соль – это случайная строка данных, которая добавляется к паролю перед его хешированием (например, **пароль + случайный_набор_символов**).

- Соль генерируется индивидуально для каждого пользователя и хранится в базе данных рядом с хешем.
- В результате злоумышленнику пришлось бы генерировать уникальную радужную таблицу под каждую отдельную соль, что экономически и технически невозможно.
- Соль скрывает тот факт, что у разных учетных записей в системе совпадают пароли.

«СОЛЕНЫЙ» ХЕШ


always same
Only Hash
Message
Digest
random salt
different!
Salted Hash
Message
u45iusfc
Digest
never the same
Salted Hash
Message
34jhd8h
Digest

ХЕШ-ФУНКЦИЯ ВЕДЁТ СЕБЯ КАК СЛУЧАЙНАЯ ФУНКЦИЯ:

- для случайно выбираемого *x* все значения ***y = h(x)*** равновероятны
- даже если ***x2*** зависит от ***x1***, ***h(x2)*** и ***h(x1)*** выглядят как статистически независимые
- при изменении одного бита в ***x*** изменяется примерно половина бит в ***h(x)***
- для произвольно выбираемого ***x*** вероятность того, что в ***h(x)*** первые ***k*** бит будут нулевые, равна $\mathbf{1/2^k}$

Представленные алгоритмы вычисления хеш-функций удовлетворяют всем требованиям, предъявляемым к криптографическим хеш-функциям, в предположении стойкости используемых блоковых шифров.

ЭЛЕКТРОННАЯ ЦИФРОВАЯ ПОДПИСЬ RSA

Абонент выбирает случайно два больших простых числа P и Q. Затем он вычисляет число

$$N = PQ. \tag{2.28}$$

(Число N является открытой информацией, доступной другим абонентам.) После этого абонент вычисляет число $\phi = (P-1)(Q-1)$ и выбирает некоторое число $d < \phi$, взаимно простое с ϕ, и по обобщенному алгоритму Евклида находит число c, такое, что

$$cd \bmod \phi = 1. \tag{2.29}$$

Пусть Алиса хочет подписать сообщение $\bar{m} = m_1, \ldots, m_n$. Тогда вначале она вычисляет так называемую хеш-функцию

$$y = h(m_1, \ldots, m_n),$$

которая ставит в соответствие сообщению $\bar{m}$ число y. Предполагается, что алгоритм вычисления хеш-функции всем известен.

Алиса вычисляет число

$$s = y^c \bmod N, \tag{4.1}$$

т.е. она возводит число y в свою секретную степень. Число s это и есть цифровая подпись. Она просто добавляется к сообщению $\bar{m}$, и тем самым Алиса имеет сформированное подписанное сообщение

$$\langle \bar{m},\ s \rangle . \tag{4.2}$$

Теперь каждый, кто знает открытые параметры Алисы, ассоциированные с ее именем, т.е. числа N и d, может проверить подлинность ее подписи. Для этого необходимо, взяв подписанное сообщение (4.2), вычислить значение хеш-функции $h(\bar{m})$, число

$$w = s^d \bmod N \tag{4.3}$$

и проверить выполнение равенства $w = h(\bar{m})$.

Утверждение 4.1. *Если подпись подлинная, то* $w = h(\bar{m})$.

Доказательство. Из (4.3), (4.1) и свойств схемы RSA (см. разд. 2.6) следует

$$w = s^d \bmod N = y^{cd} \bmod N = y = h(\bar{m}). \qquad \square$$

Утверждение 4.2. *Описанная электронная подпись удовлетворяет всем требованиям, предъявляемым к подписи.*

Пример 4.1. Пусть $P = 5$, $Q = 11$. Тогда $N = 5 \cdot 11 = 55$, $\phi = 4 \cdot 10 = 40$. Пусть $d = 3$. Такой выбор d возможен, так как $\gcd(40, 3) = 1$. Параметр $c = 3^{-1} \bmod 40$ вычисляем с помощью обобщенного алгоритма Евклида (см. разд. 2.3), $c = 27$.

Пусть, например, Алиса хочет подписать сообщение $\bar{m} = abbbaa$, для которого значение хеш-функции равно, скажем, 13:

$$y = h(abbbaa) = 13.$$

В этом случае Алиса вычисляет по (4.1)

$$s = 13^{27} \bmod 55 = 7$$

и формирует подписанное сообщение

$$\langle abbbaa,\ 7 \rangle .$$

Теперь тот, кто знает открытые ключи Алисы $N = 55$ и $d = 3$, может проверить подлинность подписи. Получив подписанное сообщение, он заново вычисляет значение хеш-функции

$$h(abbbaa) = 13$$

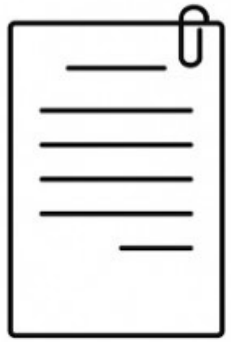
(если содержание сообщения не изменено, то значение хеш-функции совпадет с тем, которое вычисляла Алиса) и вычисляет по (4.3)

$$w = 7^3 \bmod 55 = 13.$$

Значения w и хеш-функции совпали, значит, подпись верна. $\square$

Боб подписывает документ своим секретным ключом d

Документ



Хэш документа

Хэш-функция

h = a8ad3ffd...

Боб



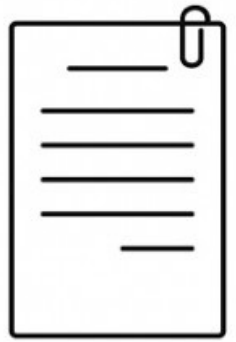
d - закрытый ключ

$s = h^d \bmod n$

s - подпись

(e, n) - открытый ключ

Документ



Подпись





Открытый ключ

Алиса проверяет подпись документа открытыми ключами Боба (e, n)

Документ



Хэш документа

Хэш-функция

h = a8ad3ffd...

Алиса



(e, n) - открытый ключ

s - подпись

$h' = s^e \bmod n$

h = h'?

Подписал Боб

ЭЛЕКТРОННАЯ ПОДПИСЬ НА БАЗЕ ШИФРА ЭЛЬ-ГАМАЛЯ

Алиса собирается подписывать документы. Она выбирает большое простое число p и число g, такие, что различные степени g суть различные числа по модулю p. Эти числа передаются или хранятся в открытом виде и могут быть общими для целой группы пользователей.

Алиса выбирает случайное число x, $1 < x < p - 1$, которое она держит в секрете. Это ее секретный ключ, только она его знает. Затем она вычисляет число

$$y = g^x \bmod p. \qquad (4.4)$$

Это число y Алиса публикует в качестве своего открытого ключа. Заметим, что при больших p, зная y, невозможно найти x (это задача дискретного логарифмирования).

Теперь Алиса может подписывать сообщения. Допустим, она хочет подписать сообщение $\bar{m} = m_1, \ldots, m_n$. Опишем последовательность действий для построения подписи.

Вначале Алиса вычисляет значение хеш-функции $h = h(\bar{m})$, которое должно удовлетворять неравенству $1 < h < p$. Затем Алиса выбирает случайно число k ($1 < k < p-1$), взаимно простое с $p-1$, и вычисляет число

$$r = g^k \bmod p. \tag{4.5}$$

Далее Алиса вычисляет числа

$$u = (h - xr) \bmod (p-1), \tag{4.6}$$

$$s = k^{-1}u \bmod (p-1). \tag{4.7}$$

Под k^{-1} в (4.7) подразумевается число, удовлетворяющее уравнению

$$k^{-1}k \bmod (p-1) = 1. \tag{4.8}$$

Такое k^{-1} существует, так как k и $p-1$ взаимно просты, и может быть найдено по обобщенному алгоритму Евклида. Наконец, Алиса формирует подписанное сообщение

$$\langle \bar{m};\ r,\ s \rangle. \tag{4.9}$$

Получатель подписанного сообщения (4.9), прежде всего, заново вычисляет значение хеш-функции $h = h(\bar{m})$. Затем он проверяет подпись, используя равенство

$$y^r r^s = g^h \bmod p. \tag{4.10}$$

Утверждение 4.3. *Если подпись верна, то условие* (4.10) *выполняется.*

Доказательство. Действительно,

$$y^r r^s = (g^x)^r \left(g^k\right)^s = g^{xr} g^{k\left(k^{-1}(h-xr)\right)} = g^{xr} g^h g^{-xr} = g^h \bmod p.$$

(Здесь первое равенство следует из (4.4) и (4.5), второе из (4.7).)

Утверждение 4.4. *Описанная электронная подпись удовлетворяет всем требованиям, предъявляемым к подписи.*

П р и м е р 4.2. Пусть общие параметры для некоторого сообщества пользователей $p = 23$, $g = 5$. Алиса выбирает свой секретный ключ $x = 7$ и вычисляет открытый ключ y по (4.4):

$$y = 5^7 \bmod 23 = 17.$$

Пусть Алиса создала документ $\bar{m} = baaaab$ и хочет его подписать.

Перейдем к вычислению подписи по алгоритму. Прежде всего она вычисляет хеш-функцию, пусть ее значение $h(\bar{m}) = 3$. Затем Алиса генерирует случайное число k, например, $k = 5$. Вычисления по (4.5), (4.6) дают

$$r = 5^5 \bmod 23 = 20,$$

$$u = (3 - 7 \cdot 20) \bmod 22 = 17.$$

Далее Алиса находит $k^{-1} \bmod 22$:

$$k^{-1} \bmod 22 = 5^{-1} \bmod 22 = 9.$$

Вычисления по (4.7) дают

$$s = 9 \cdot 17 \bmod 22 = 21.$$

Наконец, Алиса формирует подписанное сообщение в виде (4.9):

$$\langle baaaab,\ 20,\ 21 \rangle.$$

Подписанное сообщение передается, Боб его получает и проверяет подлинность подписи. Вначале он вычисляет значение хеш-функции

$$h(baaaab) = 3,$$

затем вычисляет левую часть (4.10)

$$17^{20} \cdot 20^{21} \bmod 23 = 16 \cdot 15 \bmod 23 = 10$$

и после этого правую часть (4.10)

$$5^3 \bmod 23 = 10.$$

Боб делает вывод, что подпись верна.

Рассмотренный метод электронной подписи сложнее, чем RSA, а его стойкость базируется на другой, нежели в RSA, односторонней функции. Это важно для криптографии, так как в случае дискредитации одного метода можно использовать другой. Кроме того, на основе подписи Эль-Гамаля может быть построен более эффективный алгоритм, в котором время вычислений значительно сокращается за счет использования «коротких» показателей степени. Такой алгоритм представлен в следующем разделе.

СЕМЕЙСТВО ЭЦП ГОСТ

- **ГОСТ Р 34.10-94** – Самый первый стандарт ЭЦП, базировался на сложности вычисления дискретного логарифма. **Отменен**.
- **ГОСТ Р 34.10-2001** – Переход на эллиптические кривые. Обеспечивал длину ключа 256 бит. **Отменен**.
- **ГОСТ 34.10-2012** – **Действующий стандарт**. Определяет схемы подписи с длиной ключа 256 бит (уровень стойкости до 2026-2030 гг.) и 512 бит (для долгосрочной защиты). Именно на нем сейчас выпускаются подписи ФНС и для Госуслуг.

ХЕШИРОВАНИЕ В ГОСТ

- **ГОСТ Р 34.11-94** – Старый алгоритм (размер хэша 256 бит), тесно связанный с ГОСТ 28147-89. **Отменен**.
- **ГОСТ 34.11-2012 («Стрибог» / Streebog)** – **Действующий стандарт**. Разработан с нуля, выдает хэш-код длиной 256 или 512 бит. Построен по схеме классической функции перестановки.

СЕМЕЙСТВО ЭЦП DSS (DIGITAL SIGNATURE STANDART)

История версий стандарта FIPS 186

1. **FIPS 186 (1994 год):**
   - Ввел базовый алгоритм **DSA**, основанный на трудности вычисления дискретных логарифмов в конечных полях.
   - Поддерживал хеш-функцию **SHA-1**.

2. **FIPS 186-1 (1998 год) и FIPS 186-2 (2000 год):**
   - Добавил алгоритм на эллиптических кривых — **ECDSA** (Elliptic Curve Digital Signature Algorithm) и алгоритм **RSA**.

3. **FIPS 186-3 (2009 год):**
   - Увеличил требования к длине ключей и привел спецификации ECDSA и RSA в соответствие с современными криптографическими требованиями.

СЕМЕЙСТВО ЭЦП DSS (DIGITAL SIGNATURE STANDART)

История версий стандарта FIPS 186

4. **FIPS 186-4 (2013 год):**
   - Закрепил поддержку семейства алгоритмов: классический **DSA**, **ECDSA** и **RSA**. Рекомендовал использовать для DSA размер ключа от 2048 бит и выше.

5. **FIPS 186-5 (февраль 2023 года — актуальная версия):**
   - **DSA исключен из создания новых подписей** и оставлен исключительно для проверки старых (legacy) документов.
   - В состав одобренных алгоритмов входят **RSA**, **ECDSA** и новый современный алгоритм **EdDSA** (Edwards Curve Digital Signature Algorithm на кривых Ed25519 и Ed448).

СТАНДАРТЫ НА ЭЛЕКТРОННУЮ (ЦИФРОВУЮ) ПОДПИСЬ. ГОСТ Р34.10-94. FIPS 186. (DSS)

Вначале для некоторого сообщества пользователей выбираются общие несекретные параметры. Прежде всего необходимо найти два простых числа, q длиной 256 бит и p длиной 1024 бита, между которыми выполняется соотношение

$$p = bq + 1 \tag{4.11}$$

для некоторого целого b. Старшие биты в p и q должны быть равны единице. Затем выбирается число $a > 1$, такое, что

$$a^q \bmod p = 1. \tag{4.12}$$

В результате получаем три общих параметра — p, q и a.

Далее каждый пользователь выбирает случайно число x, удовлетворяющее неравенству $0 < x < q$, и вычисляет

$$y = a^x \bmod p. \tag{4.13}$$

Число x будет секретным ключом пользователя, а число y — открытым ключом.

Пусть имеется сообщение $\bar{m}$, которое необходимо подписать. Генерация подписи выполняется следующим образом:

1. Вычисляем значение хеш-функции $h = h(\bar{m})$ для сообщения m, значение хеш-функции должно лежать в пределах $0 < h < q$ (в российском варианте хеш-функция определяется ГОСТом Р34.11-94).

2. Формируем случайное число k, $0 < k < q$.

3. Вычисляем $r = \left(a^k \bmod p\right) \bmod q$. Если оказывается так, что $r = 0$, то возвращаемся к шагу 2.

4. Вычисляем $s = (kh + xr) \bmod q$. Если $s = 0$, то возвращаемся к шагу 2.

5. Получаем подписанное сообщение $\langle \bar{m};\ r,\ s \rangle$.

Для проверки подписи делаем следующее.

1. Вычисляем хеш-функцию для сообщения $h = h(\bar{m})$.

2. Проверяем выполнение неравенств $0 < r < q,\ 0 < s < q$.

3. Вычисляем $u_1 = s \cdot h^{-1} \bmod q,\ u_2 = -r \cdot h^{-1} \bmod q$.

4. Вычисляем $v = (a^{u_1} y^{u_2} \bmod p) \bmod q$.

5. Проверяем выполнение равенства $v = r$.

Если хотя бы одна из проверок на шагах 2 и 5 не дает нужного результата, то подпись считается недействительной. Если же все проверки удачны, то подпись считается подлинной.

Утверждение 4.5. *Если подпись к сообщению была сформирована законно, т.е. обладателем секретного ключа* x, *то* $v = r$.

Доказательство. Запишем следующую цепочку равенств, которая следует непосредственно из описания метода (напомним, что показатели степени приводятся по модулю q):

$$\begin{aligned} v &= \left(a^{sh^{-1}} y^{-rh^{-1}} \bmod p\right) \bmod q = \\ &= \left(a^{(kh+xr)h^{-1}} a^{-xrh^{-1}} \bmod p\right) \bmod q = \\ &= \left(a^{k+xrh^{-1}-xrh^{-1}} \bmod p\right) \bmod q = \\ &= \left(a^{k} \bmod p\right) \bmod q = r. \end{aligned}$$

З а м е ч а н и е. Чтобы найти параметр a, удовлетворяющий (4.12), рекомендуется использовать следующий метод. Берем случайное число $g > 1$ и вычисляем

$$a = g^{(p-1)/q} \bmod p. \tag{4.14}$$

Если $a > 1$, то это то, что нам нужно. Действительно, на основании (4.14) и теоремы Ферма имеем

$$a^q \bmod p = g^{((p-1)/q)q} \bmod p = g^{p-1} \bmod p = 1,$$

т.е. выполняется равенство (4.12). Если при вычислении по (4.14) мы получаем $a = 1$ (крайне маловероятный случай), то нужно просто взять другое число g.

Пример 4.3. Выберем общие несекретные параметры

$$q = 11, \quad p = 6q + 1 = 67,$$

возьмем $g = 10$ и вычислим

$$a = 10^6 \bmod 67 = 25.$$

Выберем секретный ключ $x = 6$ и вычислим открытый ключ

$$y = 25^6 \bmod 67 = 62.$$

Сформируем подпись для сообщения $\bar{m} = baaaab$. Пусть для хеш-функции этого сообщения $h(\bar{m}) = 3$. Возьмем случайно число $k = 8$. Вычислим

$$r = \left(25^8 \bmod 67\right) \bmod 11 = 24 \bmod 11 = 2,$$
$$s = (8 \cdot 3 + 6 \cdot 2) \bmod 11 = 36 \bmod 11 = 3.$$

Получаем подписанное сообщение

$$\langle baaaab;\ 2,\ 3\rangle.$$

Теперь выполним проверку подписи. Если сообщение не изменено, то $h = 3$. Вычислим

$$
\begin{aligned}
h^{-1} &= 3^{-1} \bmod 11 = 4, \\
u_1 &= 3 \cdot 4 \bmod 11 = 1, \\
u_2 &= -2 \cdot 4 \bmod 11 = -8 \bmod 11 = 3, \\
v &= \left(25^1 \cdot 62^3 \bmod 67\right) \bmod 11 = \\
&= (25 \cdot 9 \bmod 67) \bmod 11 = 24 \bmod 11 = 2.
\end{aligned}
$$

Мы видим, что $v = r$, значит подпись верна.

Теперь остановимся на отличиях американского стандарта от российского. Они сводятся к следующему.

1. Длина числа q берется равной 160 бит.
2. В качестве хеш-функции используется алгоритм SHA-1.
3. При генерации подписи на шаге 4 параметр s вычисляется по формуле $s = k^{-1}(h + xr) \bmod q$.
4. При проверке подписи на шаге 3 u_1 и u_2 вычисляются по формулам $u_1 = h \cdot s^{-1} \bmod q$, $u_2 = r \cdot s^{-1} \bmod q$.

С учетом этих отличий нетрудно переписать всю схему подписи в «американском» стиле. Доказательство корректности алгоритма проводится совершенно аналогично.